Repoute

오픈소스 기여 온보딩 AI Agent

광주 3반 G082 손서현

핵심 기능

- **파일 읽는 순서 추천**

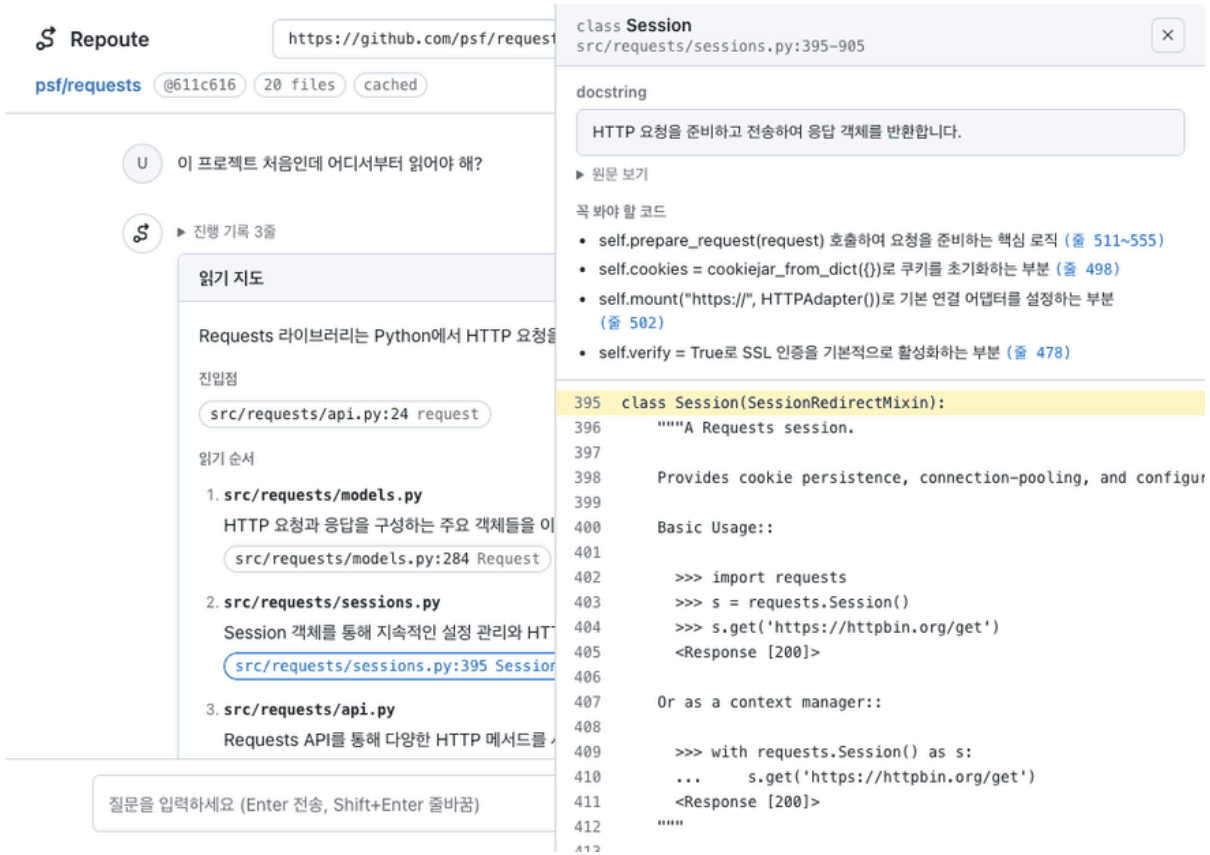
  읽기 시작할 첫 파일과 읽기 순서
- **함수 호출 순서 추적**

  동작 질문 → 실행 순서대로 코드와 설명
- **관심사 기반 이슈 추천**

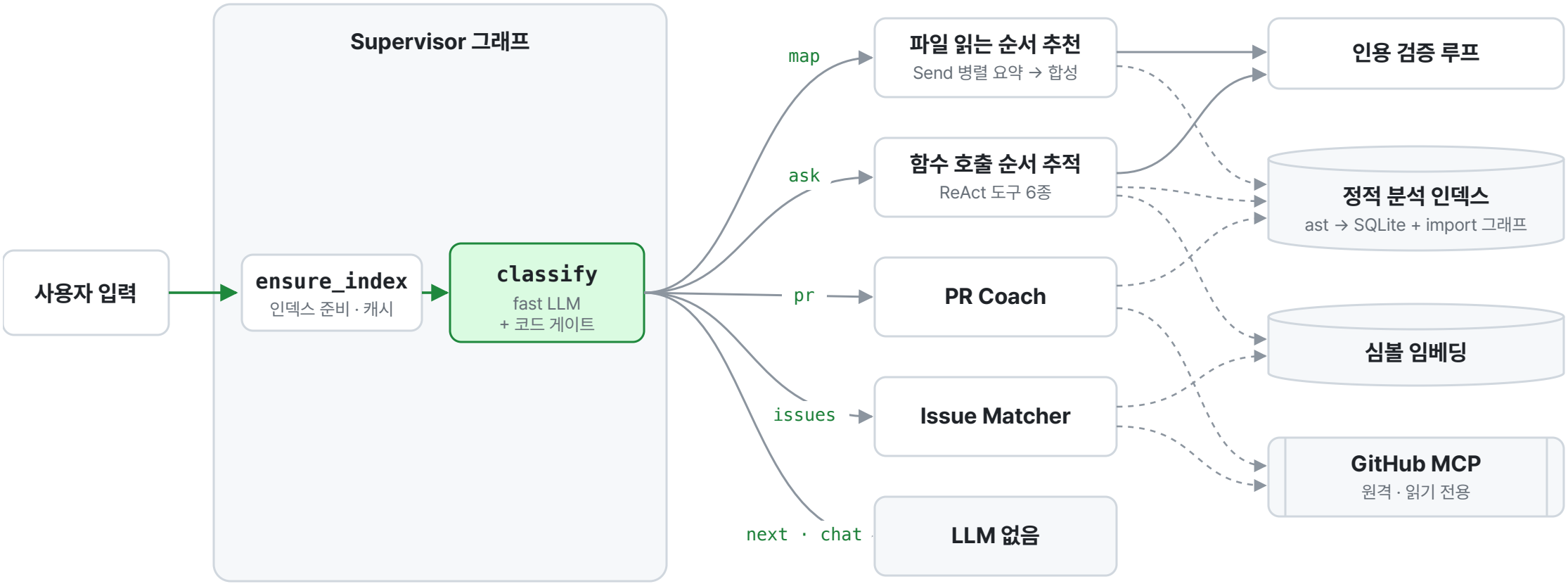
  관심 분야 · 수준에 맞는 열린 이슈
- **PR 리뷰 확인**

  바뀐 함수 · 영향 범위 · 확인할 점
- **코드 경로 클릭**

  해당 코드 + docstring 한국어 요약 + 꼭 확인할 코드 줄



아키텍처


Supervisor 그래프
사용자 입력
ensure_index
인덱스 준비 · 캐시
classify
fast LLM
+ 코드 게이트
map
ask
pr
issues
next · chat
파일 읽는 순서 추천
Send 병렬 요약 → 합성
함수 호출 순서 추적
ReAct 도구 6종
PR Coach
Issue Matcher
LLM 없음
인용 검증 루프
정적 분석 인덱스
ast → SQLite + import 그래프
심볼 임베딩
GitHub MCP
원격 · 읽기 전용

Multi-agent

Supervisor 그래프

- 인덱스 준비 → 의도 분류 → 4개로 분기 → 대화 기억

파일 읽는 순서 Send 병렬

- 하나씩 요약하면 오래 걸림
- 파일 요약을 동시에 실행 → 한 번에 합침

함수 호출 순서 ReAct

- 한 번에 답을 알 수 없음
- 함수 찾기 · 코드 열기를 반복하며 근거 수집

이슈 추천 Human-in-the-loop

- 관심 분야 · 수준을 모르면 멈추고 질문
- 답을 받으면 멈춘 곳부터 이어서 실행

```
# Supervisor 그래프 · 인덱스 준비 → 의도 분류 → 기능 실행 → 대화 기억
builder.add_edge(START, "ensure_index")
builder.add_edge("ensure_index", "classify")
builder.add_conditional_edges("classify", route, ROUTES)  # 4개로 분기
builder.add_edge(node, "remember")
graph = builder.compile(checkpointer=checkpointer)  # 대화 상태 저장

# 서브 에이전트
add_conditional_edges("select_targets", fan_out)  # Send 병렬
add_edge("tools", "agent")                        # ReAct 반복
interrupt({"kind": "profile", ...})               # 멈추고 질문
```

MCP

- GitHub 원격 MCP 서버 연결
- 이슈 추천 → 열린 이슈 목록

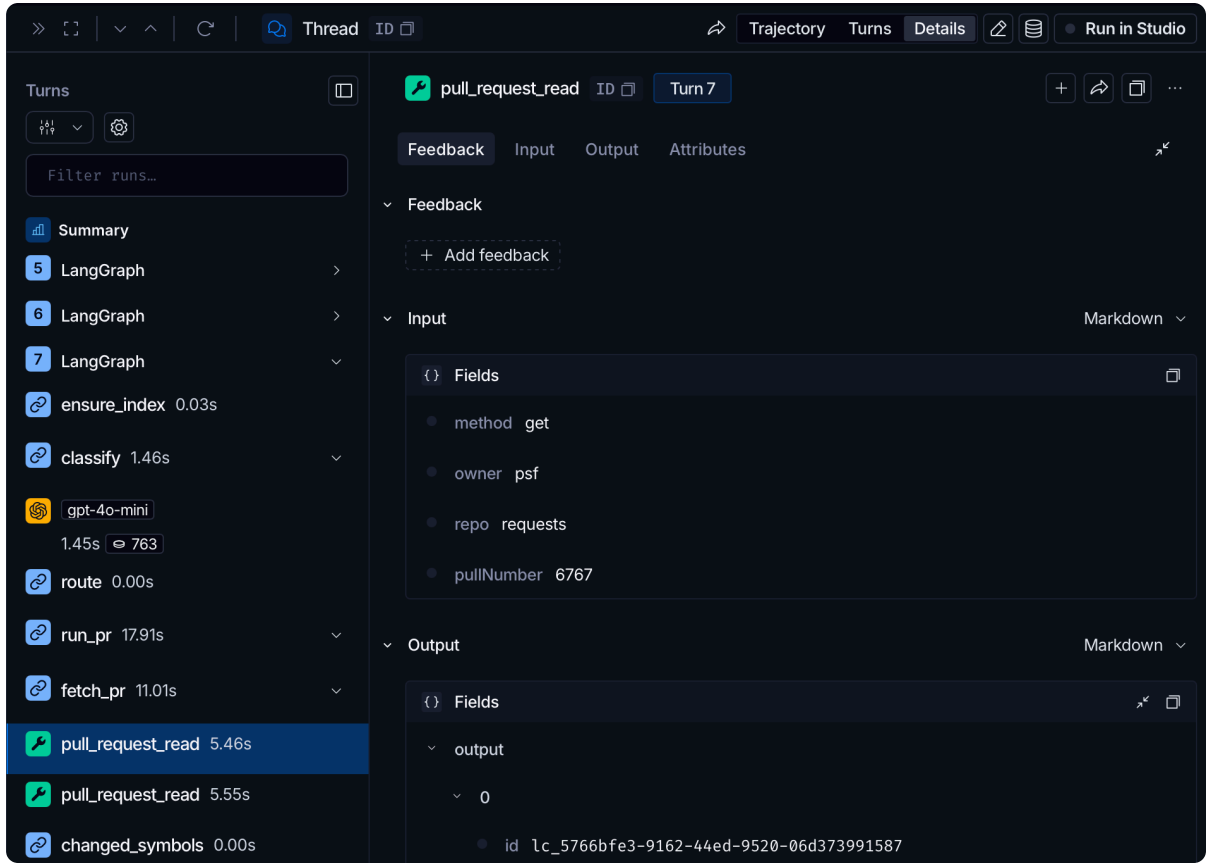
  PR 리뷰 → PR 내용 · 바뀐 파일
- 읽기 전용 묶음 `x/issues/readonly` 에서

  쓰는 툴만 남김
- LLM 이 아니라 코드가 정해진 인자로 직접 호출

```
url = GITHUB_MCP_URL + f"x/{toolset}"      # 읽기 전용 묶음
tools = filter_tools(tools, ISSUE_TOOL_NAMES)  # 쓰는 툴만
call_tool_sync(tool, args)                 # 코드가 직접 호출
```

툴 스키마 (이름 · 하는 일 · 입력값 형식)

12.8k → **1.2k 토큰** (−91%)



RAG

사용처

- 함수 이름 없는 질문의 출발 함수 찾기
- 이슈와 관련된 코드 찾기

저장

- 함수 하나 = 임베딩 한 건

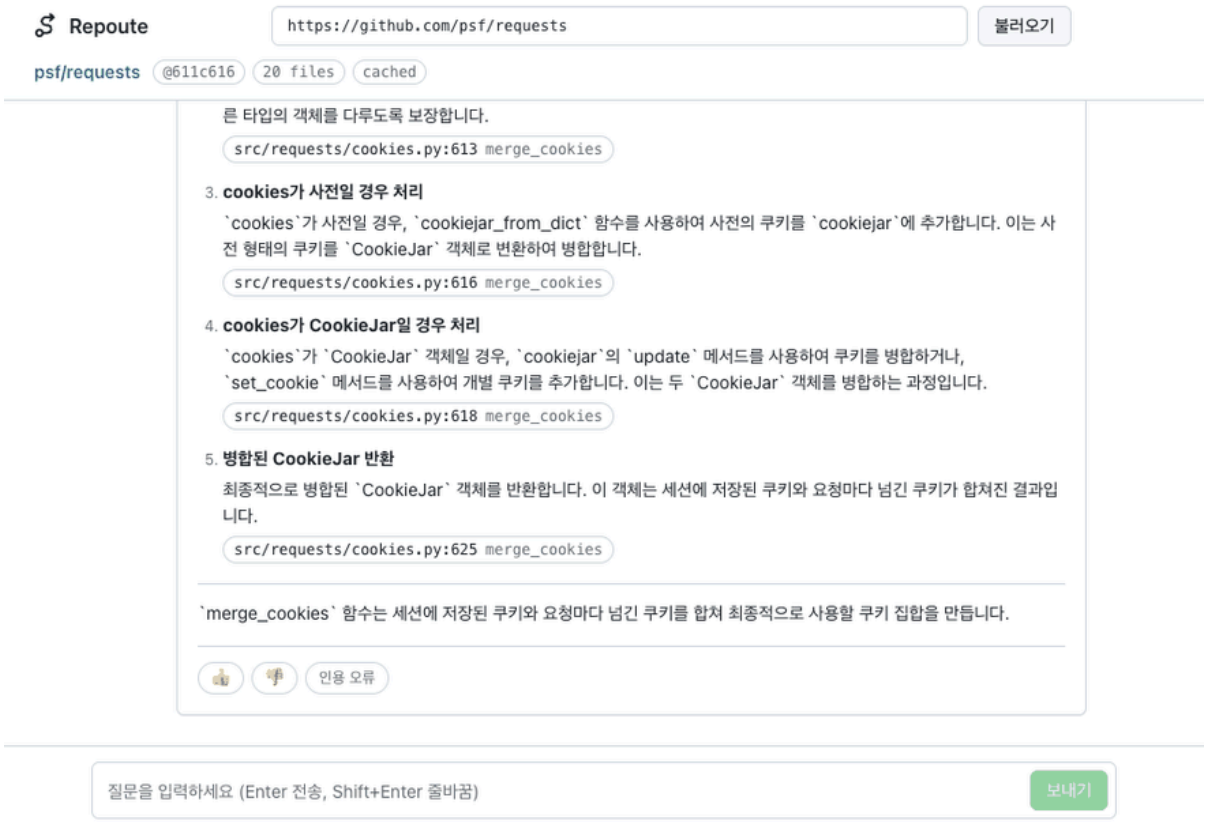
  종류 · 이름 · 정의 줄 · 설명 · 파일

- 같은 커밋이면 다시 만들지 않음

검색

- 질문 → 의미가 비슷한 함수 → 코드 열어 보고 근거로 답

```
f"{kind} {name}\n{def 줄}\n{docstring}\n{file}"  # 저장 단위
scores = matrix @ query                          # 검색
```



의존성 주입

- LLM · 인덱스 · 임베딩 · MCP 도구를 노드 안에서 만들지 않고 Deps 로 묶어 밖에서 주입
- 테스트에는 가짜 LLM, 실제 서버에는 진짜 LLM → 같은 코드로 실행

모듈화

- LLM 쓰는 층 / 안 쓰는 층(정적 분석 · 근거 검증) 분리

단위 테스트

- 테스트 파일 37개 전부 통과 · 가짜 LLM 으로 비용 0

```python
@dataclass
class Deps:
    fast_llm: Any              # 저비용 모델
    quality_llm: Any           # 고품질 모델
    index: RepoIndex           # 정적 분석 결과
    embedder: Any = None

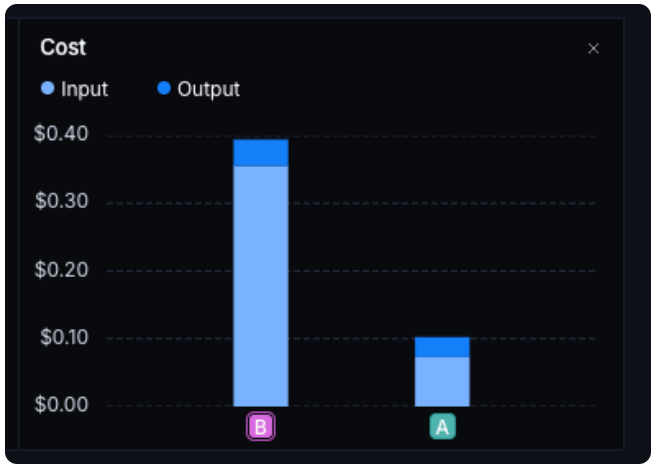
# 노드에 미리 넣어 두기
builder.add_node("classify",
                 partial(classify, deps=deps))
```

수치 개선

항목	개선 전	개선 후	방법
핵심 파일 정확도	0.32	**0.72**	import 관계로 파일 순위를 매기는 방식 개선
근거 정확도	65%	**100%**	코드를 읽지 않으면 답하지 못하게 막음
함수 이름 없는 질문	0 / 10	**6 / 10**	RAG 검색 추가
MCP 툴 스키마	12.8k	**1.2k**	읽기 전용 툴 중 쓰는 것만 남김

모델 비교 같은 질문 5개, 분홍 quality / 초록 fast



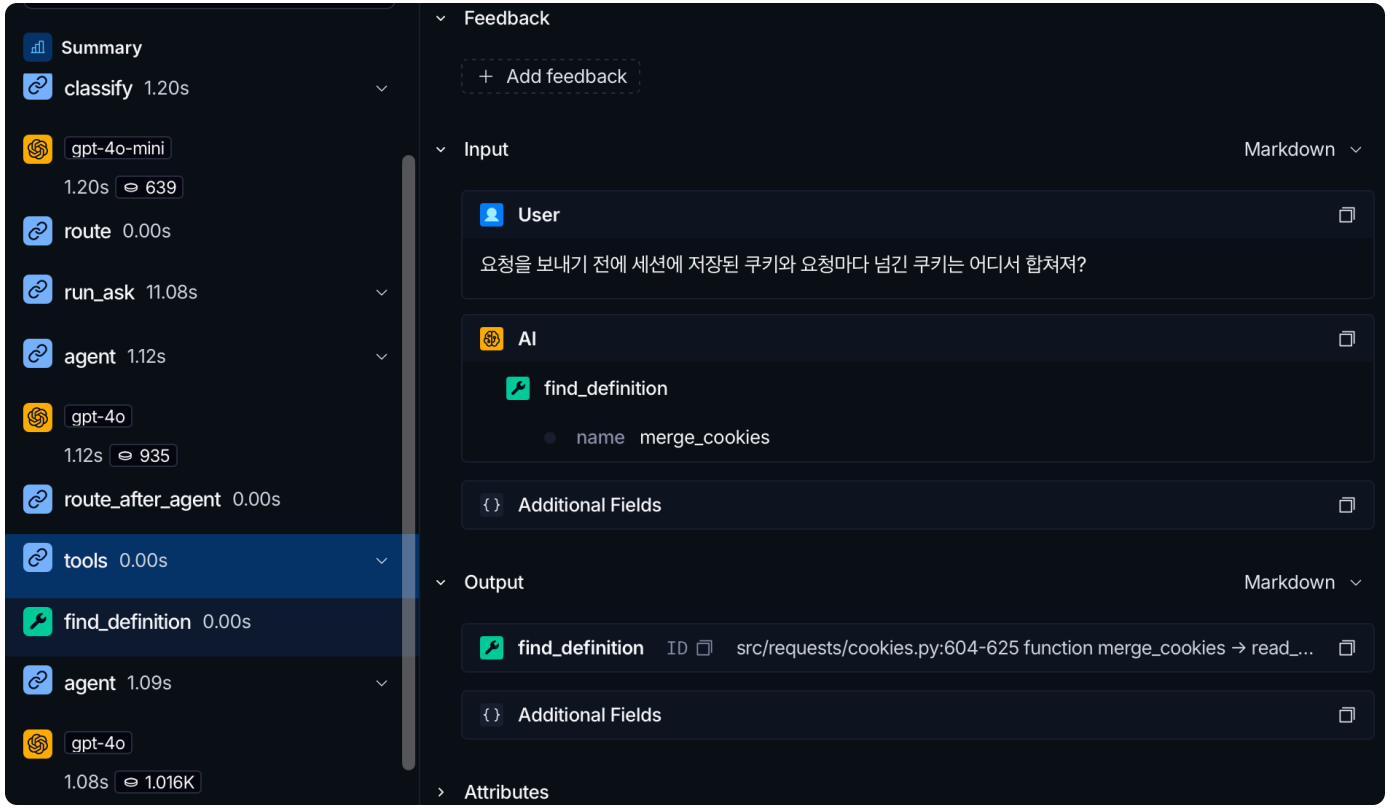


비용은 약 4배 늘었지만 정답 도달이 0.2에서 0.6으로 올라 quality 모델을 선택했습니다.

LangSmith

- **트레이싱** 모든 LLM · 도구 호출 기록 — Send 병렬, ReAct 반복 확인
- **Experiment** 같은 데이터로 개선 전후 비교 — 진입점 정확도 0.32 → 0.53, 비용 거의 같음
- **Feedback** 사용자 평가를 실행에 연결 · **CI 회귀 게이트** 코드 바뀔 때마다 정확도 측정





한계 · 개선 방향

구분	내용	대응 · 개선 방향
한계	Python 레포만 지원	tree-sitter 로 다른 언어 지원
한계	평가 데이터가 작음 (레포 5개 · 질문 20개)	평가 데이터 확대 · 신규 기여자의 첫 PR 까지 효과 검증
트레이드오프	quality 모델: 정답 도달 3배 ↔ 비용 4배	끝까지 답하는 것을 우선해 quality 선택
트레이드오프	RAG 도입: 첫 실행 임베딩 비용 · 시간	커밋 단위 캐시 → 같은 커밋은 비용 0
트레이드오프	ReAct 반복 호출: 무한 반복 가능성	도구 15회 · 재작성 2회 상한

느낀점

레포 전체를 LLM에 넣으면 비용과 시간이 감당이 안 돼서, 정적 분석으로 필요한 부분만 골라 넘기고 모델도 역할별로 나누는 구조를 설계하는 것이 가장 어려웠습니다.
그 과정에서 LLM 비용은 호출 횟수보다 무엇을 얼마나 넘기느냐에 달려 있다는 것을 배웠습니다.